

普通克里金 GPU 空间插值

Ordinary Kriging on CUDA — 代码理论详解

本文档解释 `kriging_cuda.cu` 这段程序背后的数学原理与 GPU 并行策略。程序用 CUDA 在显卡上完成**普通克里金 (Ordinary Kriging)** 空间插值：给定一批散布的采样点，估计整个网格上每个位置的数值。

1 什么是克里金插值

克里金是一种**地统计学 (Geostatistics)** 插值方法。它的核心思想是：某个未采样位置 x_0 的数值，可以用已知的 n 个采样点数值的**加权平均**来估计：

$$z^*(x_0) = \sum_i w_i \cdot z_i \quad \text{且} \quad \sum_i w_i = 1$$

与简单的反距离加权 (IDW) 不同，克里金的权重 w_i 不是凭经验设定的，而是**根据数据本身的空间相关结构解出来的**，因此它是统计最优（无偏且方差最小）的线性估计。约束条件 $\sum w_i = 1$ 保证估计是**无偏的**。

2 半变异函数 (Semivariogram)

克里金用**半变异函数** $\gamma(h)$ 来描述"两点相距 h 时，数值差异有多大"。距离越近，数值越相似 (γ 越小)；距离越远，差异越大 (γ 趋于平台值)。代码里 `variogram()` 函数实现了三种常见模型：

模型	公式 (psill = sill - nugget)	特点
球状 Spherical	nugget + psill $\cdot$ (1.5r - 0.5r ³), $r=h/\text{range}$; 超过 range 后取 sill	最常用，达到 range 后完全平坦
指数 Exponential	nugget + psill $\cdot$ (1 - e ^{-3h/range})	渐近逼近 sill，平滑
高斯 Gaussian	nugget + psill $\cdot$ (1 - e ^{-3r²})	近距离极平滑，适合连续场

三个关键参数：

- nugget (块金)**： $h \rightarrow 0$ 时的 γ 值，代表测量噪声或微尺度变化。

- **sill (基台)** : γ 的平台值, 即数据的总方差。
- **range (变程)** : 达到基台的距离, 超过此距离两点几乎不相关。

关键性质: $\gamma(0) = 0$ 。代码里 `if (h <= 0) return 0;` 正是这一点, 它保证了克里金是**精确插值器**——在采样点上会原样还原实测值。

3 克里金方程组的矩阵结构

要解出权重, 需要建立并求解一个 $(n+1) \times (n+1)$ 的线性方程组 $A \cdot s = b$:

$$A = \begin{bmatrix} G & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \quad b = \begin{bmatrix} g_0 \\ \mu \end{bmatrix}$$

$$s = \begin{bmatrix} w \\ \mu \end{bmatrix}$$

各部分含义:

- $G[i][j] = \gamma(|x_i - x_j|)$: 采样点**两两之间**的半变异, 只与采样点位置有关。
- $g_0[i] = \gamma(|x_i - x_0|)$: 每个采样点到**目标点**的半变异。
- μ : **拉格朗日乘子**, 用来强制满足 $\sum w_i = 1$ 这个无偏约束。矩阵最外圈的那一行/一列全 1 (右下角为 0) 就是这个约束的体现。

4 核心优化: 矩阵 A 只求逆一次

这是整个程序高效的关键。观察矩阵结构会发现:

- A **只依赖采样点的位置**, 与目标点无关 $\rightarrow$ 对所有网格点都是同一个 A。
- 每个目标点变化的**只有右端向量 b**。

因此程序在主机 (CPU) 端用**高斯-约当消元法 (带部分主元)** 把 A **求逆一次** (`invertMatrix()`), 得到 A^{-1} 。之后每个网格点只需做矩阵-向量乘法即可解出权重。正是这一点让问题在整个输出网格上"高度并行" (embarrassingly parallel), 非常适合 GPU。

数值精度: 代码用 `double` (双精度) 保持克里金方程组的良态。若矩阵奇异 (`best < 1e-14`), 通常是采样点**坐标重复**, 需要去重或加一点 nugget。

5 克里金方差

克里金不仅给出估计值，还能给出这个估计的**不确定度**，即克里金方差：

$$\sigma^2(x_0) = b^T \cdot A^{-1} \cdot b$$

这个方差的物理意义很直观：**在采样点附近，方差接近 0**（我们很确定）；**远离数据的地方，方差增大**（我们更不确定）。代码在采样点处会把因舍入产生的微小负值钳制为 0。

6 两个 GPU 核函数

程序提供了两个内核，对应“要不要方差”的取舍：

	krigeExactKernel（精确）	krigeFastKernel（快速）
并行粒度	一个 CUDA 线程块处理一个目标点	一个线程处理一个目标点
输出	估计值 + 克里金方差	仅估计值
每点复杂度	$O(n^2)$	$O(n)$
关键手法	块内构造 b ，做 $A^{-1}b$ 与块内归约	预计算 $v = A^{-1}[z;0]$ ，直接点乘

快速内核的巧妙之处

由于 A 是对称矩阵，估计值可以改写为：

$$z^*(x_0) = [z;0]^T A^{-1} b = v^T b$$

其中 $v = A^{-1}[z;0]$ 在主机端**预先算好一次**。这样每个目标点就不用再解方程组，只需把 v 和 b 点乘，复杂度从 $O(n^2)$ 降到 $O(n)$ ，速度大幅提升——代价是拿不到方差。快速内核还用**共享内存分块（tiling）**把采样点流式载入，减少全局内存访问。

7 GPU 并行与共享内存要点

- **block 分工**：精确内核里每个块负责一个目标点，块内线程协作构造 b 、做矩阵乘和归约（要求 `blockDim` 是 2 的幂）。
- **共享内存**：精确内核用共享内存存放 b 和归约暂存区；若采样点太多会超出设备共享内存上限，此时需减少采样数或改用“移动搜索邻域”。

- **避免过早 return：**快速内核里用 `alive` 标志而非 `return`，因为块内后面有 `__syncthreads()`，线程提前退出会导致同步死锁。

8 验证的正确性属性

程序结尾会验证几个克里金必须满足的性质，可作为结果是否正确的判据：

- 权重之和恒为 1（无偏约束成立）。
- 在采样点上原样还原实测值（精确插值）——代码检查 `max error at sample locations` 应接近 0。
- 采样点处克里金方差约为 0，远离数据处方差增大。
- 精确内核与快速内核的估计值应几乎一致（`max |exact - fast|` 极小）。

输出文件：`kriging_output.csv`（含每个网格点的估计值与方差）与 `kriging_estimate.pgm`（估计场的灰度快视图）。程序把运行日志重定向到 `log.txt`。